Reinforcement learning from human feedback

SESSION 3 · REINFORCEMENT LEARNING FOR LANGUAGE MODELS

Pegah Alizadeh

pegahani.github.io

pegah.alizadeh@univ-lyon2.fr

Master's course, 2026–2027

Last session

- **Bradley-Terry:** $P(y_w \succ y_l \mid x) = \sigma\left(r(x, y_w) - r(x, y_l)\right)$, fitted by maximum likelihood, a logistic regression
- **A reward model** is Bradley-Terry with a neural network as the utility: a language model with a linear head
- **The lab:** our reward model beat a "longer wins" rule, but it also **learned the length bias** in its labels

Today: InstructGPT's step 3. We have a reward; how do we train the model to get more of it, without breaking it?

Today

1. **The RLHF objective:** reward, KL divergence, and why both
2. **Using a reward model without RL:** rejection sampling and best-of-N
3. **Why RL?**
4. **Policy gradients:** the intuition, then the derivation, step by step
5. **Wiring RL into RLHF:** the training loop, the reference model, the KL penalty
6. **REINFORCE and RLOO**
7. **PPO:** clipping, the value function, GAE
8. **GRPO and its variants**, and the details that change the objective
9. **RL for reasoning**, then the **lab**

Today's main sources: Lambert, *RLHF*, Chapters 3, 6, 7 and 9. Lambert's course: Lecture 3, Lecture 4.

The RLHF objective

The RLHF objective

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \quad \underbrace{r_\phi(x,y)}_{\text{maximize the reward}} \quad - \underbrace{\beta D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\right)}_{\text{but don't change the model too much}}$$

- **Red: learned.** θ, the parameters of the policy π_θ we train today. **Green: given.** Fixed before training starts.
- π_{ref} is the reference model, usually the SFT model; r_ϕ is last session's reward model, now frozen; $\mathcal{D}$ is the prompt dataset
- β (we choose it) sets the trade-off between improving behavior and staying close to what the model already knows

Not every method today keeps this exact form. It's the starting point; each method changes a piece of it.

Method	Reward	KL to π_{ref}
Best-of-N, rejection sampling	r_ϕ	**no KL term**: Best-of-N doesn't train; rejection sampling is plain fine-tuning
REINFORCE, RLOO, PPO	r_ϕ	subtracted **from the reward**
GRPO	r_ϕ or a checker	a separate term **in the loss**
GSPO, CISPO, RL for reasoning	often a checker	often **dropped**, $\beta = 0$

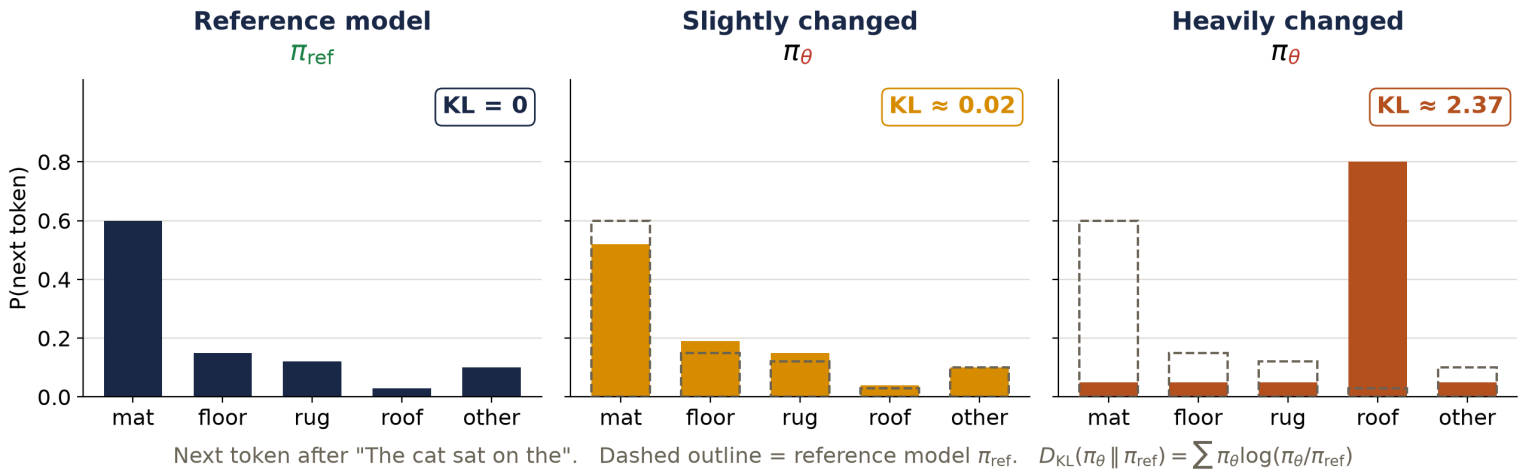
KL divergence: how far has the model moved?

Why? the objective has a KL term, a **cost**: we want the model to move some, not too much. What does KL measure?

KL measures how different two distributions are: **bigger KL = further apart.**

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i}.$$

Example: the next word after *"The cat sat on the"*, under the reference model and two changed versions of it.



It's 0 only when $p = q$ and grows as the model moves away. It's **asymmetric**, so it's **a divergence, not a metric**, even if people say "KL distance".

KL between two policies

OPTIONAL

π_θ: model being trained; π_{ref}: frozen copy from before RL (often SFT). Sum over **whole responses** y:

$$D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \sum_y \pi_\theta(y \mid x) \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} = \mathbb{E}_{y \sim \pi_\theta}\left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\right]$$

- $\pi_\theta(y \mid x)$ **is not 1:** it's a product of token probabilities, usually tiny; over all y they sum to 1.
- **Too many y to sum:** V choices for each of T tokens gives V^T responses. So we sample from π_θ and average the log ratio.

V: vocabulary size (OLMo 2: about 100K). T: max response length.

Example: x = *"The cat sat on the"*, y = the next word.

y	$\pi_\theta(y)$: how often	$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$
roof	80%	$\log \frac{0.80}{0.03} = +3.28$
mat	5%	$\log \frac{0.05}{0.60} = -2.48$
others	5% each	$-1.10,\ -0.88,\ -0.69$

Same as the figure on the previous slide: π_{ref} = reference bars, π_θ = "heavily changed" bars (made-up numbers). Averaging the last column over samples gives 2.37 = **KL**.

Which direction of KL? OPTIONAL

KL is asymmetric, so the order matters. **Forward KL** puts the target first, $D(\text{target} \,\|\, \text{model})$; **reverse KL** puts the trained model first. The RLHF penalty $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$ is a **reverse KL**.

	Reverse KL (RLHF): $D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$	Forward KL (SFT): $D_{\mathrm{KL}}(p_{\mathrm{data}} \| \pi_\theta)$
Average over	the model's own outputs	the data
Punishes	outputs the reference finds implausible (roof: 0.80×3.28)	data the model gives near-zero probability
Behavior	**mode-seeking:** may drop some options	**mass-covering:** must cover all the data

Same two distributions, both orders: $D(\text{heavily changed} \,\|\, \text{ref}) \approx 2.37$ but $D(\text{ref} \,\|\, \text{heavily changed}) \approx 1.73$.

RL algorithms also monitor $D_{\mathrm{KL}}(\pi_{\mathrm{old}} \,\|\, \pi_\theta)$ on the batch sampled from π_{old}. In each case, the expectation is under the distribution we can sample from. Source: Lambert, Chapter 15.

What changes from standard RL

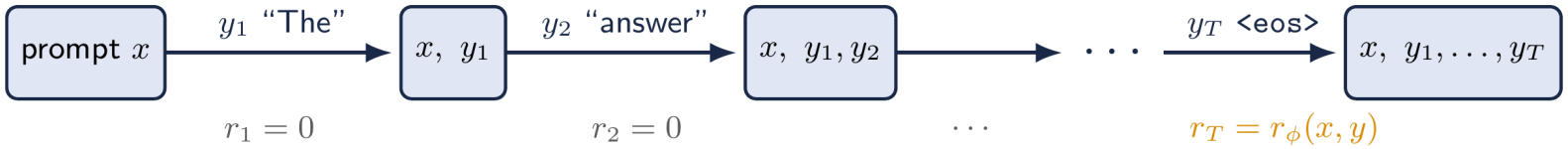
	Standard RL	RLHF for language models
Policy	Often learned from scratch	Fine-tuned from a pretrained model
Reward	Environment function $r(s_t, a_t)$	Learned model $r_\phi(x, y)$
Transitions	Unknown, random: $p(s' \mid s, a)$	Known, deterministic: x from a dataset, then append tokens
Action	One environment action	A completion y, a sequence of tokens
Reward timing	Often per step	Usually once, for the whole response
Horizon, discount	$T > 1, \gamma < 1$	Often single-step, $\gamma = 1$

Transitions: per response, there is no next state; per token, the next state is the text plus the new token (next slide).

Source: Lambert, Chapter 3.

Two views of the same problem

Token level: an MDP T decisions, reward only at the end



Response level: a contextual bandit one decision, one reward



Token level (MDP): state = prompt + tokens so far, action = next token; $\pi_\theta(y \mid x) = \prod_t \pi_\theta(y_t \mid x, y_{<t})$.

Response level (contextual bandit): context = prompt, action = whole response; $\max_\pi \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)}[r_\phi(x, y)]$.

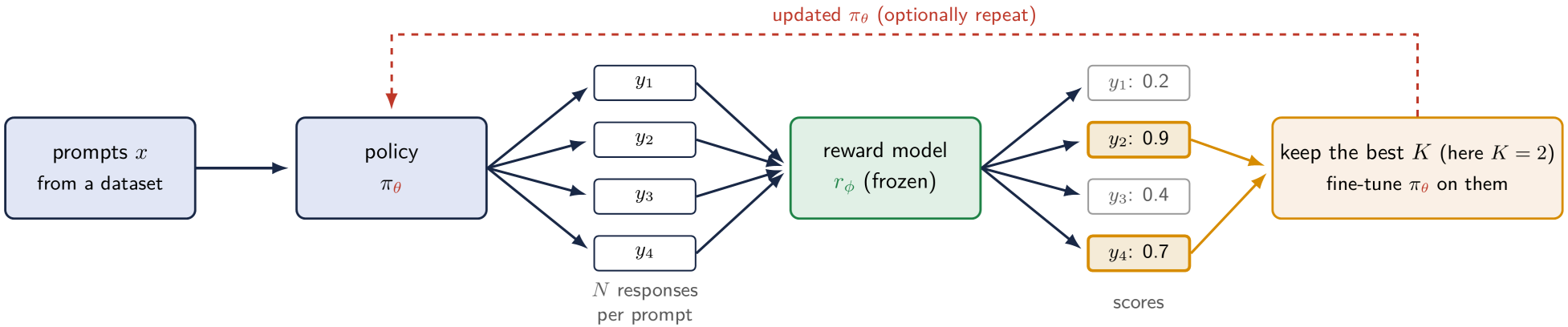
Bandit? Slot machines ("one-armed bandits"): pick an arm, get a reward, the round ends. *Contextual*: you see a context (the prompt) first (Sutton & Barto, §2.9).

Same reward, at the same time, in both. The difference is **who gets the credit**: one action, or T token actions that must share it. The methods later today differ in which view they take.

Lattimore & Szepesvári, 2020; Sutton & Barto, 2018

Using a reward model without RL

Rejection sampling



Generate N responses per prompt → **score** them with r_ϕ → **keep** the best K → **fine-tune** π_θ on them, with the instruction-tuning loss.

No policy gradient, no RL loop: it's **filtered supervised learning**. Used in WebGPT, Anthropic's helpful and harmless assistant, and Llama 2 (Bai et al., 2022) (Nakano et al., 2021) (Touvron et al., 2023).

The name comes from computational statistics: sample from a simple distribution, accept or reject each sample, and approximate a harder target.

Figure: redrawn after Lambert, Chapter 9.

Touvron et al., 2023

Selecting what to keep

OPTIONAL

With M prompts (rows) and N responses each (columns), the rewards $r_\phi(x, y)$ form a matrix $R \in \mathbb{R}^{M \times N}$:

$$R = \begin{array}{l|cccc} & \text{resp. 1} & \text{resp. 2} & \text{resp. 3} & \text{resp. 4} \\ \hline \text{prompt 1} & \mathbf{0.7} & 0.3 & 0.5 & 0.2 \\ \text{prompt 2} & 0.4 & \mathbf{0.8} & 0.6 & 0.5 \\ \text{prompt 3} & \mathbf{0.9} & 0.3 & 0.4 & 0.7 \\ \text{prompt 4} & 0.2 & 0.5 & \mathbf{0.8} & 0.6 \\ \text{prompt 5} & 0.5 & 0.4 & 0.3 & \mathbf{0.6} \end{array}$$

Two ways to keep 5 responses from this matrix:

- **Best response of each prompt** (bold): one per prompt, so every prompt is represented
- **5 highest rewards in the whole matrix** (0.9, 0.8, 0.8, 0.7, 0.7): prompt 3 gets **two** responses, prompt 5 gets **none** (its best is 0.6). Prompts where the model already scores well take more of the data.

Source: Lambert, Chapter 9, “Selection example”.

Best-of-N sampling

Best-of-N: generate N responses at inference time and return the one with the highest reward. **Nothing is trained:** neither the policy nor the reward model r_ϕ, which only picks the winner.

- A strong, simple **baseline** for RL methods
- You can still measure its KL divergence from the base model and compare it with trained policies
- It spends compute at inference instead of training, an idea that returns with reasoning models later today

Source: Lambert, Chapter 9.

Why RL?

What RL adds

OPTIONAL

Rejection sampling and best-of-N are simpler. Online RL is harder to build, but practitioners report benefits that are hard to measure:

- **It fixes rough edges:** outputs become more robust and better formatted
- **It's surgical:** RL on a narrow set of prompts tends not to damage general abilities
- **It explores:** it learns from the model's own responses, beyond a fixed dataset

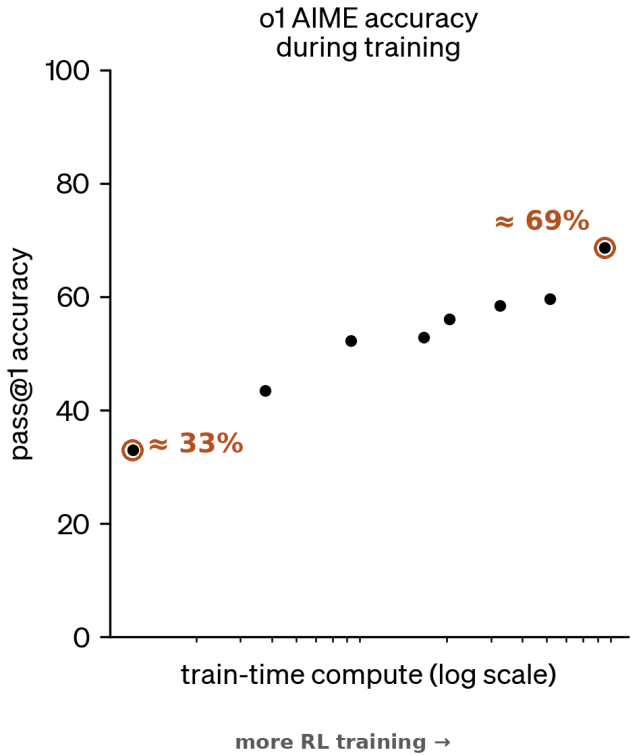
RL is now the **central step** in training the most capable models, through reasoning models such as OpenAI's o1 and DeepSeek R1 (Guo et al., 2025) (OpenAI, 2024).

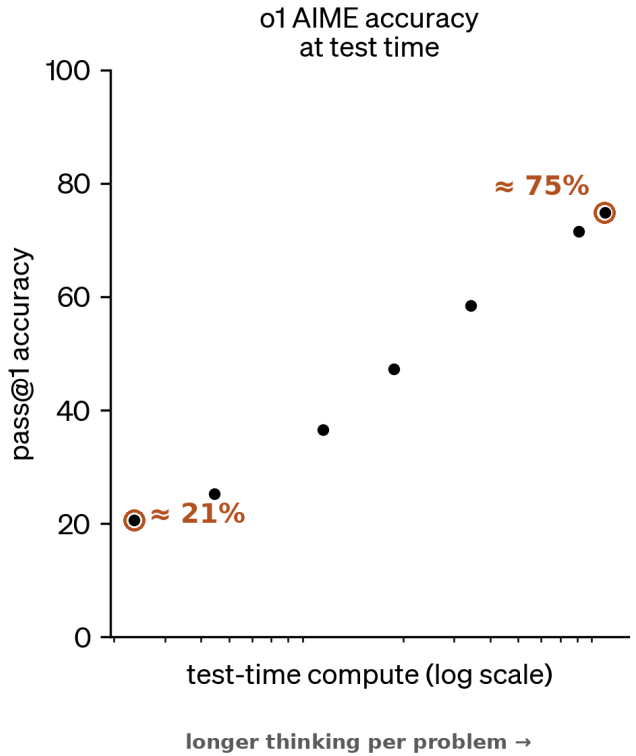
Source: Lambert, Chapters 6 and 7.

The scaling RL era

OPTIONAL

RL is now the **load-bearing step** in training the most capable models. Both plots: o1's accuracy on **AIME** (a 15-problem maths competition; **pass@1** = solved in one attempt) against compute, **log scale**, no numbers given. Source: OpenAI, *Learning to reason with LLMs*, 2024.





Left: train time. Each dot is a checkpoint during RL training. More RL: about 33% → 69%.

Right: test time. The same model, allowed to **think longer** per problem: about 21% → 75%.

Takeaway: two ways to spend compute, both give steady gains. RL is what teaches the model to use the extra thinking.

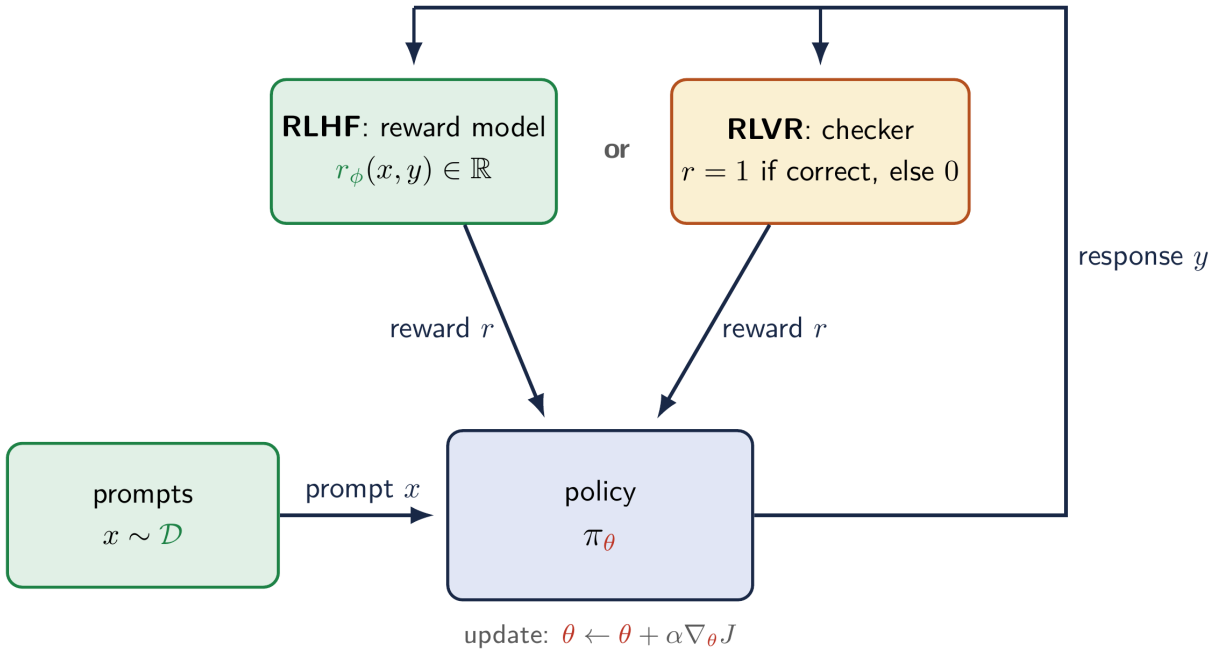
OpenAI, 2024

Same algorithms, different rewards

- **RLHF:** a **reward model** scores subjective quality
- **RL with verifiable rewards (RLVR):** a **checker** scores correctness, such as a math answer or passing tests

The policy gradient algorithms today work for both. Only the source of the reward changes.

Figure: redrawn after Lambert, Chapter 7.



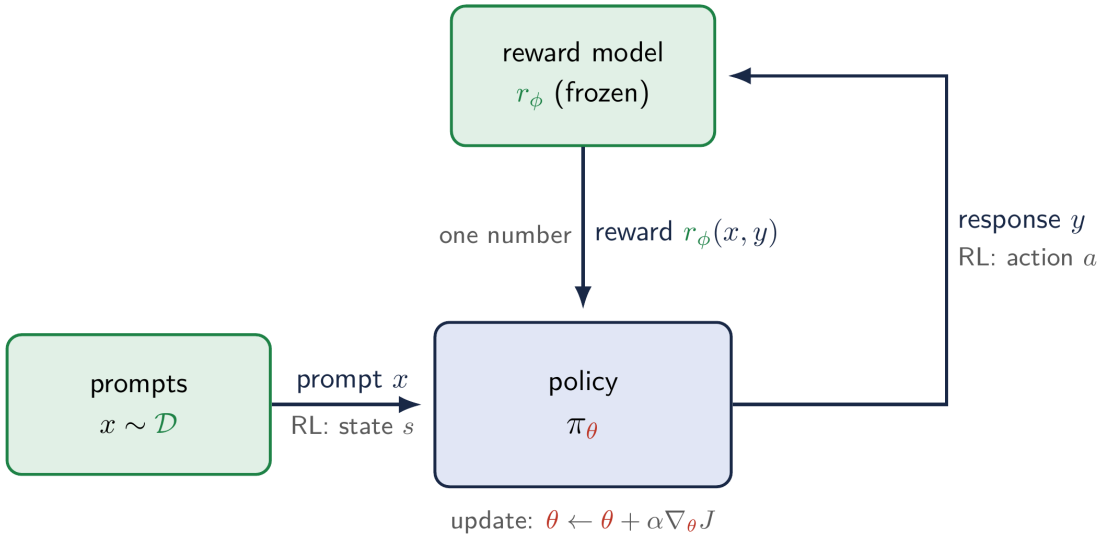
Lambert et al., 2024

Policy gradients: Core intuitions

Notation in this lecture

Why? the next slides use RL notation (s, a). Here is how it maps to ours, (x, y).

Language model	RL name	Symbol
prompt x	(initial) state	$s_0 = x$
tokens so far $x, y_{<t}$	state at step t	s_t
next token y_t	action at step t	a_t
whole response y	trajectory (episode)	τ
model $\pi_\theta(y_t \mid x, y_{<t})$	policy	$\pi_\theta(a_t \mid s_t)$
score $r_\phi(x, y)$	return (total reward)	$R(\tau)$



- **Token view:** each token is an action, $s_t \to a_t \to s_{t+1}$. **Response view:** the whole y is one action.
- In RLHF, the per-step rewards r_t are 0 except at the end, so $R(\tau) = r_\phi(x, y)$ (plus a KL term, later).
- ρ_t will denote an importance-sampling **ratio** (PPO, GRPO), never a reward.

Language models as episodic MDPs OPTIONAL

MDP concept	Language model
State s_t	Prompt + tokens generated so far: $(x, y_{<t})$
Action a_t	Next token y_t
Transition	Deterministic: append token to sequence
Policy $\pi_\theta(a_t \mid s_t)$	LM next-token distribution
Episode	One prompt → completion
Terminal reward	RM score or verifier output
γ	Typically 1.0 (no discounting)

Two views of this MDP, as at the start: **token-level** (each token is an action) or **sequence-level** (the whole completion is one action).

What are we optimizing?

Why? We want a θ whose responses get high reward **on average**, over prompts and over everything the model might write.

A **trajectory** τ for a language model is one prompt and one complete response: $\tau = (x, y_1, y_2, \ldots, y_T)$.

Its probability under the current model, $p_\theta(\tau)$, is the chance of drawing this prompt, times the chance the model writes each token:

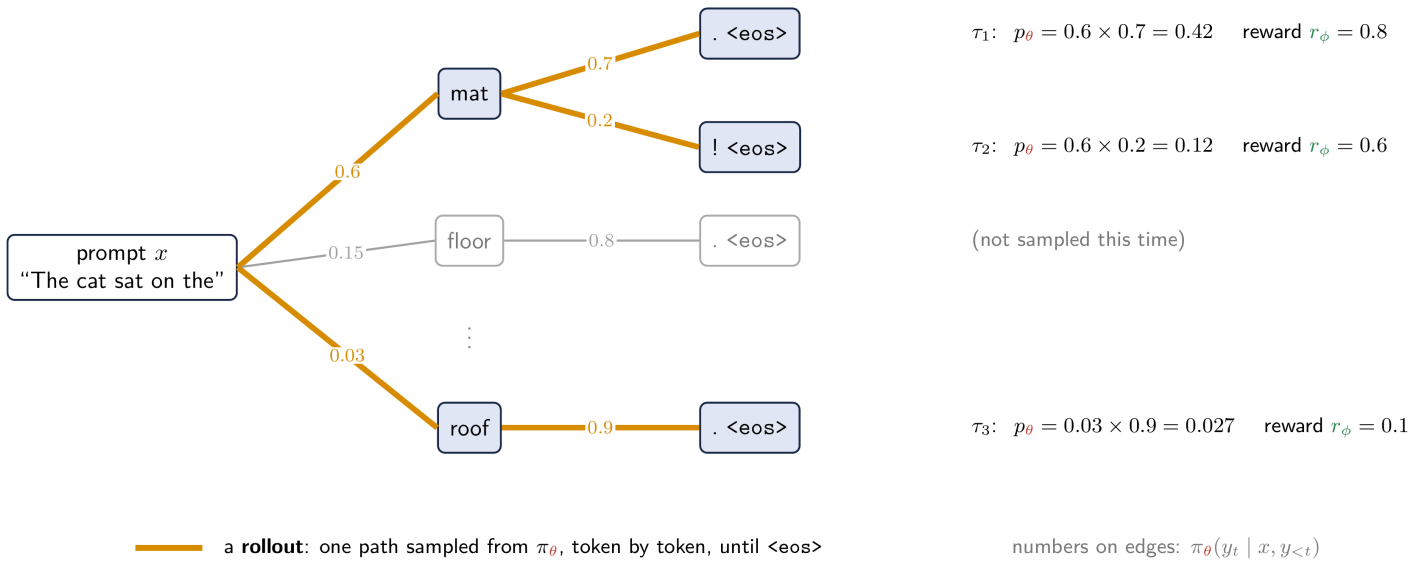
$$p_\theta(\tau) = \mathcal{D}(x) \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t})$$

No environment randomness: the next state is always "text so far + new token". So p_θ is just **the model's probability of the whole text**. The objective is the average reward under it:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)] = \sum_{\tau} p_\theta(\tau)\, R(\tau)$$

Changing θ changes which texts are likely, and so changes J. $R(\tau) = r_\phi(x, y)$ is fixed.

Rollouts: estimating J by sampling



A rollout = take a prompt, let π_θ generate token by token until `<eos>`, score the result with r_ϕ. Each rollout is one sample $\tau_i \sim p_\theta$.

We can't sum over all τ (there are V^T of them), so we average over N rollouts:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} R(\tau_i), \qquad \text{here } \tfrac{1}{3}(0.8 + 0.6 + 0.1) = 0.5$$

Likely paths (0.42) come up often, rare ones (0.027) seldom, so the plain average already weights by p_θ.

How a policy gradient works

Make actions more likely when they lead to better outcomes.

$$\Delta\theta \propto \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Why only $\log \pi$? This *is* $\nabla_\theta J$ (derived next). Only π_θ depends on θ; the reward is fixed, and Ψ_t, which may contain a value function, is just a weight.

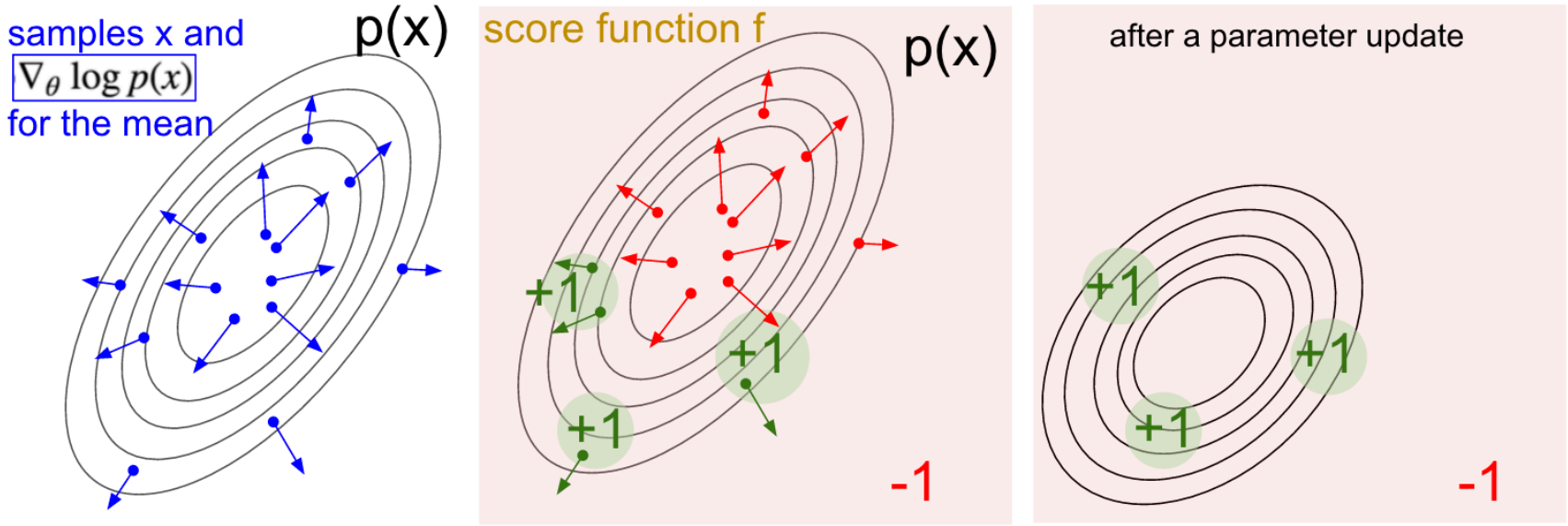
- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?** How each parameter influenced the probability of taking action a_t in state s_t. This connects the outcome back to the knobs that caused it.
- Ψ_t — **how good was it?** A scalar scoring the outcome. Positive means good, negative means bad, magnitude says how much.

> *An oversimplification (e.g. intuition in case of batch size of 1): the gradient is a vector with one entry per parameter. A positive entry means "increasing this parameter made the action more likely," a negative entry means the opposite. In practice, the update averages over many such vectors — what survives is the net vote across the batch.*

Multiply them: $\Psi_t > 0$ updates parameters to make a_t more likely, $\Psi_t < 0$ updates them to make it less likely.

The rest of this section: choosing a smart Ψ_t. All choices plug into this same update.

The policy gradient, in one picture



Left: samples x from the policy $p(x)$ (here a 2-D Gaussian). Each arrow is $\nabla_\theta \log p(x)$: the direction that makes *that* sample more likely. **Middle:** score each sample, $\Psi = +1$ (green) or -1 (red), and weight its arrow by it. Arrows of good samples are kept, bad ones are reversed. **Right:** after the update, the distribution has moved toward the $+1$ samples. **For us:** Karpathy's sample x is a whole response y, the policy is $\pi_\theta(y \mid x)$, and the score is $r_\phi(x, y)$.

Figure: Karpathy, *Deep Reinforcement Learning: Pong from Pixels*, 2016.

The policy gradient equation OPTIONAL

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

Key RL quantities

Three quantities appear throughout this lecture:

Value function $V(s)$: expected future return from state s

$$V(s) = \mathbb{E}\big[G_t \mid S_t = s\big] \quad \text{where } G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Action-value $Q(s, a)$: expected return after taking action a in state s

$$Q(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

Advantage $A(s, a) = Q(s, a) - V(s)$: how much better is action a compared to average? Positive → reinforce, negative → suppress, zero → no update.

In RLHF: often $\gamma = 1$ (no discounting) because the unit of optimization is the full completion.

What Ψ_t can be

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$G_t = \sum_{t'=t}^{T} r_{t'}$	Return from step t on ("reward-to-go")	High	None
3.	$G_t - b(s_t)$	Baselined return	Lower	None
4.	$Q^{\pi}(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^{\pi}(s_t, a_t) = Q - V$	Advantage function	Low (with good V)	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance. We'll show below that this does not change the expected gradient.

Schulman et al., 2015

Deriving the policy gradient

Setup: Differentiating an expectation

We want $\nabla_\theta J(\theta)$. We can **write** it, but not directly **estimate it from rollouts**: the distribution we sample from, p_θ, itself depends on θ.

Written as an integral, the objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)] = \int_\tau p_\theta(\tau) R(\tau)\, d\tau$$

Taking the gradient directly:

$$\nabla_\theta J(\theta) = \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau$$

We can sample trajectories from $p_\theta(\tau)$, but not from $\nabla_\theta p_\theta(\tau)$, since it is not a probability distribution.

The derivation is a few key tricks to let us approximate or compute this.

Applying the idea of computing the gradient from trajectories

First, a one-line identity from the chain rule ($\frac{d}{du}\log u = \frac{u'}{u}$):

$$\nabla_\theta \log p_\theta(\tau) = \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)}$$
$$\Rightarrow \quad \nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$$

This is the **log-derivative identity**, derived from the chain rule for $\log$.

It rewrites the gradient of a probability in terms of a log-probability, which is much easier to work with. We use it on the next slide.

Applying the idea of computing the gradient from trajectories

Start from the objective (slide 22): $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)] = \int_\tau p_\theta(\tau)\, R(\tau)\, d\tau$. Differentiate it, then substitute the log-derivative identity (previous slide):

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau && \text{differentiate the objective} \\
&= \int_\tau p_\theta(\tau) R(\tau) \nabla_\theta \log p_\theta(\tau)\, d\tau && \text{log-derivative identity (chain rule)} \\
&= \mathbb{E}_{\tau\sim p_\theta}[R(\tau)\nabla_\theta \log p_\theta(\tau)] && \text{expectation form}
\end{aligned}
$$

Why can we estimate it now? Line 1 weights R by $\nabla_\theta p_\theta$, which is not a distribution (it can be negative and sums to 0): we can't sample from it. Line 3 weights it by p_θ itself: sample rollouts, compute $R(\tau_i)\, \nabla_\theta \log p_\theta(\tau_i)$ for each, and average (Monte Carlo).

Why $\nabla_\theta p_\theta$ is not a distribution OPTIONAL

A distribution has values ≥ 0 that **sum to 1**. Its gradient sums to **0**:

$$\sum_\tau p_\theta(\tau) = 1 \quad \Rightarrow \quad \sum_\tau \nabla_\theta p_\theta(\tau) = \nabla_\theta\, 1 = 0$$

So when some responses gain probability, others must lose it. Example: take **one** parameter θ_k and increase it slightly, $\theta_k \to \theta_k + \varepsilon$. Suppose this makes the model prefer "roof" after *"The cat sat on the"*:

next word	p_θ before	after $\theta_k + \varepsilon$	change $\approx \varepsilon\, \partial p / \partial \theta_k$
mat	0.60	0.55	**−0.05**
roof	0.03	0.08	**+0.05**
others	0.37	0.37	0
sum	**1**	**1**	**0**

The first two columns are distributions: we can draw a word from them. The last one isn't: no word can be drawn "with probability −0.05". (The last column is one entry of $\nabla_\theta p_\theta$, for parameter k; the full gradient has one such column per parameter.)

Applying the idea of computing the gradient from trajectories

Where did the $p_\theta(\tau)$ term go?

It became the **sampling distribution** inside the expectation:

$$\int p_\theta(\tau) f(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta}[f(\tau)]$$

Monte Carlo then estimates that expectation by sampling trajectories from the policy:

$$\mathbb{E}_{\tau \sim p_\theta}[f(\tau)] \approx \frac{1}{N} \sum_{i=1}^{N} f(\tau_i), \qquad \tau_i \sim p_\theta$$

So no $p_\theta(\tau_i)$ factor appears in the average: **more likely trajectories already appear more often among the rollouts**.

In short: **generating rollouts** takes care of p_θ; for each rollout we only need its reward $R(\tau_i)$ and $\nabla_\theta \log p_\theta(\tau_i)$.

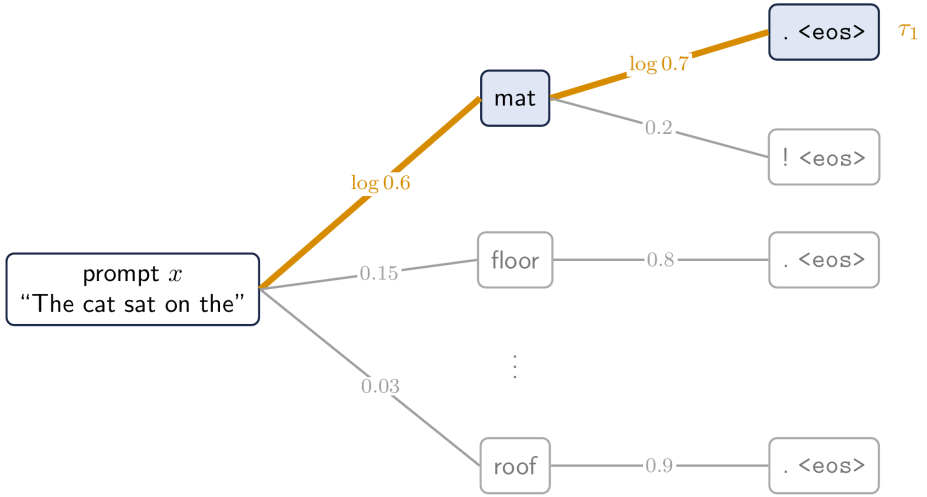
Expanding log probability of trajectory to compute ∇_θ

The trajectory probability factorizes:

$$p_\theta(\tau) = d_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t)$$

and its log is a sum:

$$\log p_\theta(\tau) = \log d_0(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$



$\log p_\theta(\tau_1) = \log \mathcal{D}(x) + \log 0.6 + \log 0.7$

For language models:

- $d_0(s_0) = \mathcal{D}(x)$: the first state is the prompt, drawn from the dataset.
- $p(s_{t+1} \mid s_t, a_t) = 1$: appending the chosen token is certain, so $\log p(s_{t+1} \mid s_t, a_t) = \log 1 = 0$.

$$\log p_\theta(\tau) = \log \mathcal{D}(x) + \sum_{t} \log \pi_\theta(y_t \mid x, y_{<t})$$

$\log \mathcal{D}(x)$ stays here; it vanishes in the gradient (next slide).

Expanding log probability of trajectory to compute ∇_θ

$$\log p_\theta(\tau) = \log d_0(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

Now take the gradient w.r.t. θ:

- $\nabla_\theta \log d_0(s_0) = 0$ — initial state distribution doesn't depend on θ
- $\nabla_\theta \log p(s_{t+1} \mid s_t, a_t) = 0$ — environment dynamics don't depend on θ
- Only $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ survives.

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

The next slide shows the same formula in code.

The policy gradient in code OPTIONAL

The gradient from the previous slide, in three lines of PyTorch (one row per response in the batch):

```
seq_log_probs = (token_log_probs * completion_mask).sum(dim=-1)
loss = -(seq_log_probs * advantages).mean()
loss.backward()
```

line	what it computes
`token_log_probs`	$\log \pi_\theta(y_t \mid x, y_{<t})$ for every token
`* completion_mask` `.sum(dim=-1)`	keep only the response tokens (not prompt or padding), add them up: $\log p_\theta(\tau)$, up to $\log \mathcal{D}(x)$
`* advantages` `.mean()`	weight each response by its Ψ, average over the batch: the Monte Carlo average
`-(...)`	optimizers **minimize**, so we minimize $-J$ to maximize J
`loss.backward()`	autodiff computes ∇_θ of the sum, i.e. the sum of the per-token gradients from the previous slide

From full return to future rewards OPTIONAL

Start from the gradient of slide 32 and plug in the sum of slide 36:

$$\begin{aligned}
\nabla_\theta J &= \mathbb{E}_{\tau\sim p_\theta}\left[R(\tau)\,\nabla_\theta \log p_\theta(\tau)\right] && \text{slide 32}\\
&= \mathbb{E}_{\tau\sim p_\theta}\left[R(\tau)\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] && \text{slide 36}\\
&= \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T} R(\tau)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] && R(\tau)\text{ inside the sum}
\end{aligned}$$

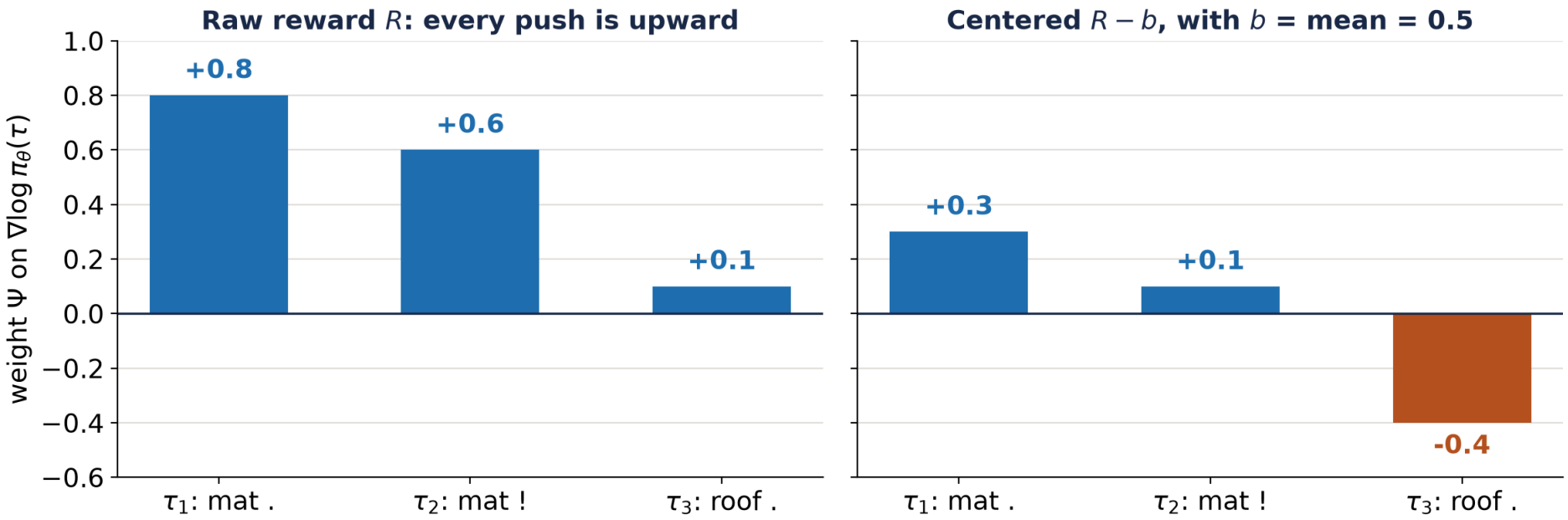
An action at time t can't affect **past** rewards. So we can replace $R(\tau)$ with the **reward-to-go** $G_t = \sum_{t'=t}^{T} r_{t'}$, only the rewards from t onward:

$$\nabla_\theta J = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T} G_t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

Same expected gradient, less noise. This is the step from Ψ_t option 1 to option 2. **In RLHF** the only reward is at the end, so $G_t = R(\tau)$ for every t: this step changes nothing there.

Why subtract a baseline?

Raw rewards are often all positive, so **every** sampled response gets pushed up, the bad ones only a bit less. Subtracting a baseline b (here the mean reward) asks instead: **better or worse than usual?** Only the better-than-usual responses go up. It doesn't change the expected gradient (proof on slide 41, optional).



Blue: make this response more likely. Orange: make it less likely. Same expected gradient, much less noise.

The law of total expectation OPTIONAL

Why? The next proof averages over actions only. This rule says why that is allowed.

$$\mathbb{E}[X] = \mathbb{E}_Y\Big[\,\mathbb{E}[X \mid Y]\,\Big]$$

To average over everything, first average **within each group**, then average the group averages, each weighted by how likely its group is. Also called the *tower rule* or *law of iterated expectations* (Wikipedia).

	School example	Next slide
everything	all students	all trajectories τ
group Y	a class	a state s, e.g. *"The cat sat on the"*
inside a group, what varies	the students of that class	the action $a \sim \pi_\theta(\cdot \mid s)$: mat, roof, …
inner average $\mathbb{E}[X \mid Y]$	the class's average grade	$\mathbb{E}_a[\,\cdot \mid s]$
outer average	weighted by class size	weighted by how often s occurs

If something is the **same for the whole group** (like $b(s)$ for a state), it can be taken out of the inner average.

Baselines keep the estimator unbiased OPTIONAL

Use a centered return, $\Psi_t = G_t - b(s_t)$.

Subtracting this baseline doesn't change the expected gradient:

$$\mathbb{E}[(G_t - b(s))\nabla_\theta \log \pi_\theta(a \mid s)] = \mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a \mid s)] - \mathbb{E}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)]$$

The first term is the original estimator. The second vanishes. The expectation is over trajectories; split it (law of total expectation, previous slide): **first fix the state** s, then average over the **action** the model picks there, $a \sim \pi_\theta(\cdot \mid s)$. $b(s)$ depends only on s, so it comes out of the inner average:

$$\mathbb{E}_\tau[b(s)\nabla_\theta \log \pi_\theta(a \mid s)] = \mathbb{E}_s\Big[\, b(s) \underbrace{\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[\nabla_\theta \log \pi_\theta(a \mid s)\big]}_{\text{inner average, over actions}} \Big] \qquad \text{total expectation}$$

$$\text{inner average} = \sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \sum_a \nabla_\theta \pi_\theta(a \mid s) \qquad \text{log-derivative identity}$$

$$= \nabla_\theta \underbrace{\textstyle\sum_a \pi_\theta(a \mid s)}_{=1} = 0$$

Example: s = *"The cat sat on the"*, $a \in$ {mat, roof, …}: whatever $b(s)$ is, it multiplies 0. So we can subtract **any function of the state**. This is why Ψ_t options 3–5 reduce variance without introducing bias.

Recall: What Ψ_t can be

OPTIONAL

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$G_t = \sum_{t'=t}^{T} r_{t'}$	Return from step t on ("reward-to-go")	High	None
3.	$G_t - b(s_t)$	Baselined return	Lower	None
4.	$Q^{\pi}(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^{\pi}(s_t, a_t) = Q - V$	Advantage function	Low (with good V)	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance; we just showed it adds no bias.

Schulman et al., 2015

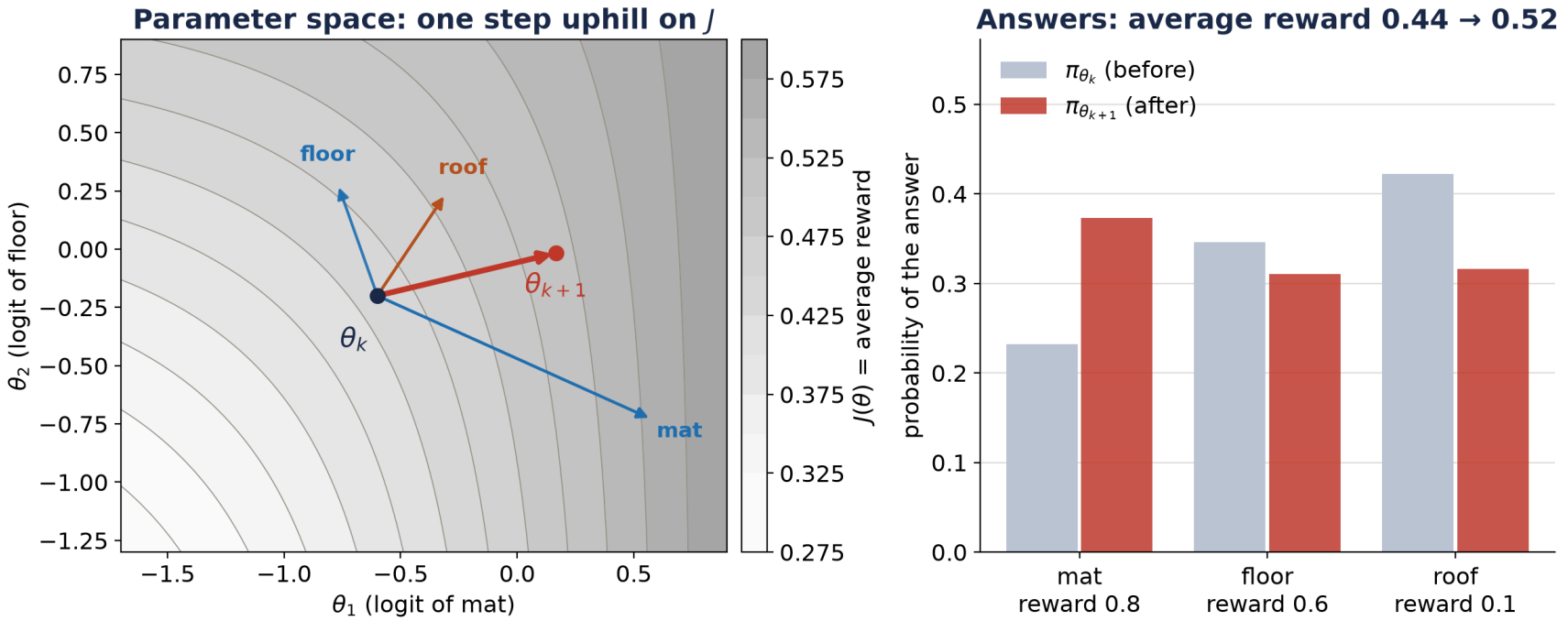
Policy gradient derivation recap

OPTIONAL

$$
\begin{aligned}
J(\theta) &= \mathbb{E}_{\tau\sim p_\theta}[R(\tau)] && \text{objective}\\
&= \int_\tau p_\theta(\tau)R(\tau)\,d\tau && \text{integral form}\\
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau)R(\tau)\,d\tau && \text{differentiate}\\
&= \mathbb{E}_{\tau\sim p_\theta}[R(\tau)\,\nabla_\theta \log p_\theta(\tau)] && \text{log-derivative identity}\\
&= \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T}\Psi_t\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] && \text{final estimator}
\end{aligned}
$$

- Log-derivative trick turns $\nabla p_\theta(\tau)$ into a sampleable expectation
- Expanding $\log p_\theta(\tau)$ gives a sum of policy log-prob terms
- Ψ_t is the learning signal: return, advantage, or TD-style estimate. **Every algorithm in this lecture is this formula with a different Ψ_t** (the *policy gradient theorem*)
- In code: sample rollouts, sum token log-probs, weight by Ψ_t, average

One policy-gradient step, in a picture



Thin arrows: one per rollout, $\Psi_i \nabla_\theta \log \pi_\theta(y_i)$ (blue: above-average reward, orange: below). Thick red arrow: their average $\hat{g}$, times the step size: $\theta_{k+1} = \theta_k + \alpha \hat{g}$.

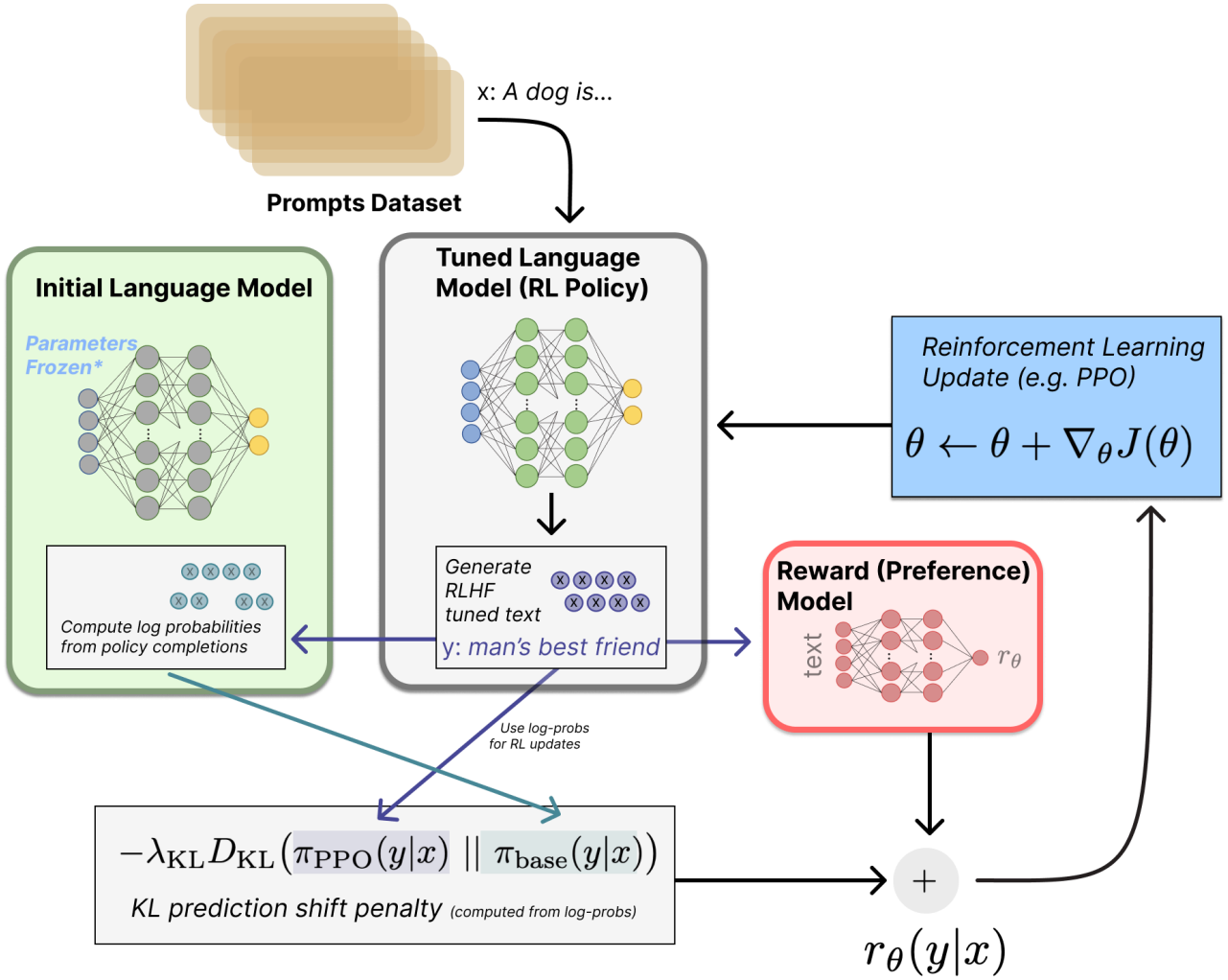
A toy policy over three answers to *"The cat sat on the"*, with two parameters, so $J(\theta)$ can be drawn (left). 6 rollouts, one thin arrow each: $\Psi_i \nabla_\theta \log \pi_\theta(y_i)$. Their average $\hat{g}$ points uphill; θ takes one step. Right: "mat" (high reward) becomes more likely, "roof" (low reward) less; the average reward rises from 0.44 to 0.52.

Wiring RLHF for language models

The RLHF training loop

Before the specific algorithms, the full RLHF setup they plug into.

Figure notation (Lambert): reward model r_θ = our r_ϕ; λ_{KL} = our β; π_{PPO} = our π_θ; π_{base} = our π_{ref}.



The reference model

OPTIONAL

The **reference model** π_{ref} is a frozen copy of the policy at the start of RL training (typically the SFT checkpoint).

It serves one purpose: **anchor the policy so it doesn't drift too far during optimization.**

Without it, the policy can exploit the reward model — finding high-scoring outputs that are degenerate or repetitive (reward hacking).

KL divergence as regularization

OPTIONAL

The KL penalty measures how far the current policy has drifted from the reference at each token:

$$\mathrm{KL}_t = \log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)$$

In practice:

1. **Generate** a completion from π_θ
2. **Compute log-probs** of those same tokens under both π_θ and π_{ref}
3. **Subtract** to get the per-token KL

The shaped reward, for intermediate tokens ($t < T$) and the final token ($t = T$):

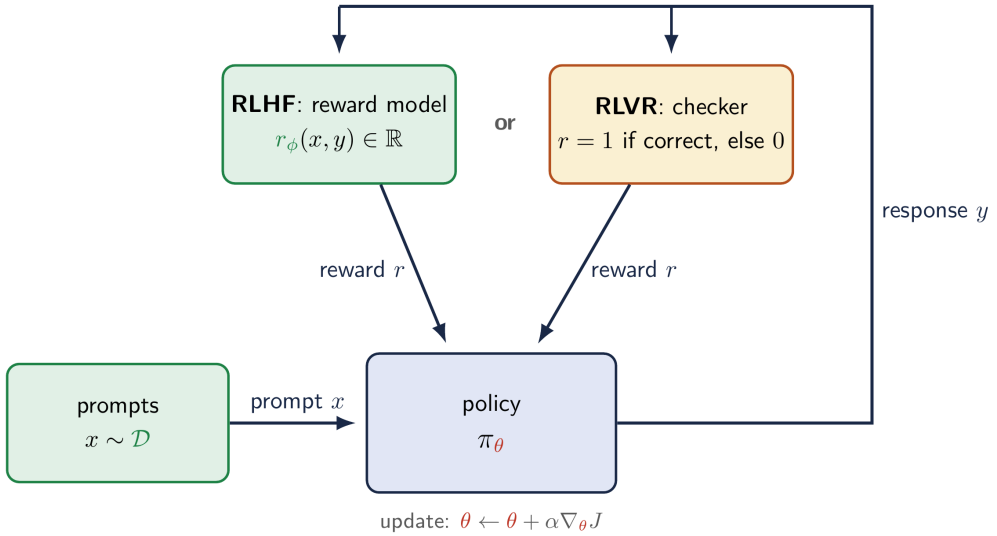
$$\tilde{r}_t = \begin{cases} -\beta\,\mathrm{KL}_t & t < T \\ r_\phi(x, y) - \beta\,\mathrm{KL}_T & t = T \end{cases}$$

KL in RLHF vs. RLVR

OPTIONAL

Why? The KL penalty protects against a reward that can be fooled. How much protection we need depends on the reward.

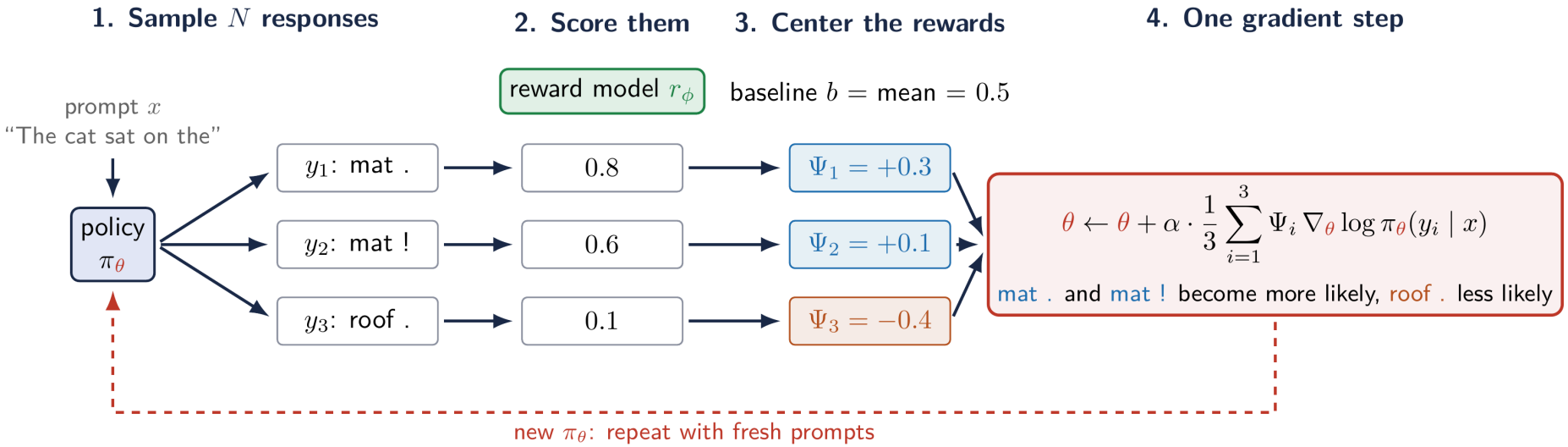
	RLHF	RLVR
reward	learned model r_ϕ	checker: correct or not
can it be fooled?	**yes**: it's an imperfect proxy	hardly: it checks the answer
KL penalty	**essential**	often small, or $\beta = 0$
reference model	needed	sometimes dropped (e.g. DeepSeek R1), saving memory



Takeaway: the more trustworthy the reward, the less KL regularization we need.

REINFORCE and RLOO

REINFORCE



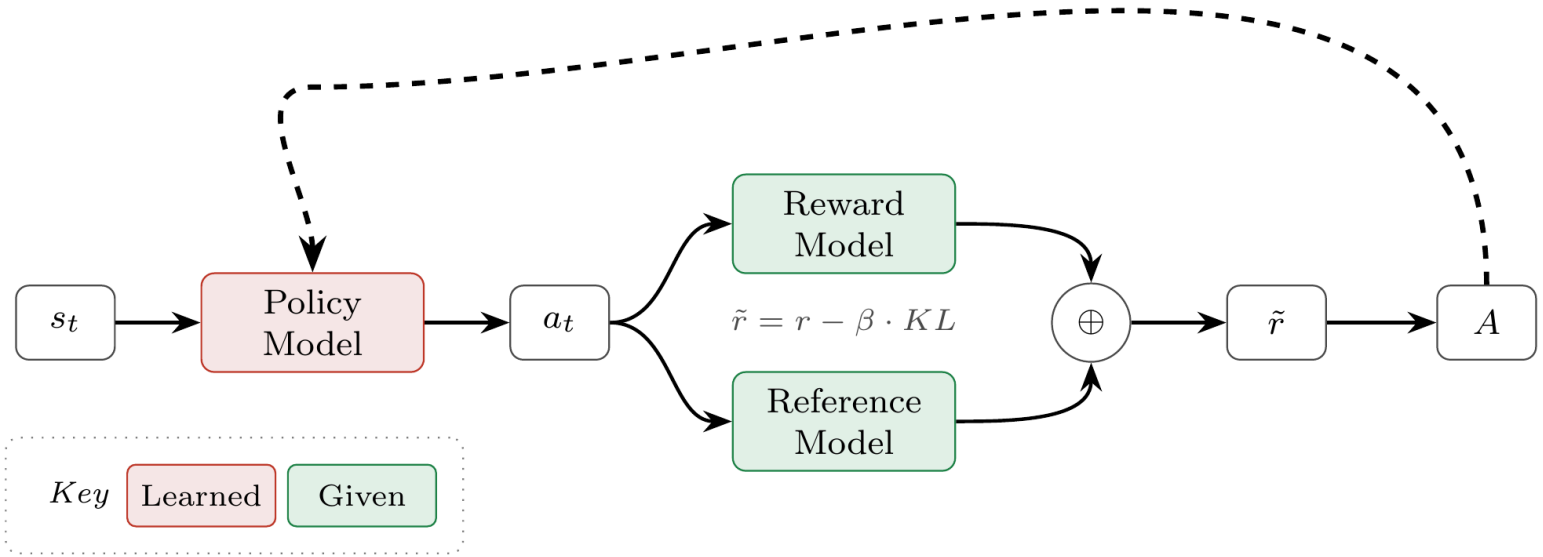
Why? It is the policy gradient we derived, used directly: sample rollouts, weight each one's $\nabla \log \pi$ by how good it was, average, step. No extra model.

For a language model, the weight of response y_i is $\Psi_i = r_\phi(x, y_i) - b$, the same for all its tokens:

$$\Delta\theta = \alpha \cdot \frac{1}{N} \sum_{i=1}^{N} \big(r_\phi(x, y_i) - b\big) \nabla_\theta \log \pi_\theta(y_i \mid x)$$

The name (Williams, 1992): **RE**ward **IN**crement = **N**onnegative **F**actor (α) × **O**ffset **R**einforcement ($r - b$) × **C**haracteristic **E**ligibility ($\nabla \log \pi$).

The common pipeline: REINFORCE



Why? REINFORCE, RLOO, PPO and GRPO all run this same loop. Each one changes **a few boxes**; the next slide says which.

1. **Sample:** the policy π_θ (learned) writes a response a_t for the prompt s_t.
2. **Score:** the reward model r_ϕ and the reference model π_{ref} (both given) give the shaped reward $\tilde{r} = r - \beta\,\mathrm{KL}$.
3. **Weight:** turn $\tilde{r}$ into an advantage A, "how much better than expected" (here: minus a baseline).
4. **Update:** one gradient step on π_θ, weighted by A (dashed arrow).

Figure: recoloured from Lambert, Chapter 6.

What each method changes in the pipeline

Why? To see the four methods as small edits of one loop, rather than four separate algorithms.

	responses per prompt	advantage A (box 3)	extra learned model	KL penalty	clipping
REINFORCE	1 or more	$\tilde{r}$ minus a baseline (e.g. batch mean)	none	in the reward	no
RLOO	K	$\tilde{r}_i$ minus the mean of the **other** $K-1$	none	in the reward	no
PPO	1 or more	per token, from a **value model** (GAE*)	value model V_ψ	in the reward, per token	**yes**
GRPO	G	$(r_i - \mu)/\sigma$ within the group	none	**in the loss**	**yes**

Each later "architecture" slide shows the same pipeline with these boxes changed.

*GAE = **generalized advantage estimation**: combines the rewards with the value model's predictions to give **each token** its own advantage (PPO section).

RLOO: Leave-one-out baseline

Why? a good baseline should know how hard the prompt is, without training an extra model. Sample several responses per prompt, and let the others be the baseline.

Key idea: generate K completions per prompt. Use the other $K - 1$ rewards as the baseline:

$$b(s, a_k) = \frac{1}{K-1} \sum_{i=1,\, i \neq k}^{K} R(s, a_i)$$

The advantage for completion k:

$$\hat{A}(s, a_k) = R(s, a_k) - b(s, a_k)$$

This is a **per-prompt** baseline that naturally captures prompt difficulty — hard prompts get low rewards across all completions, so the baseline is low.

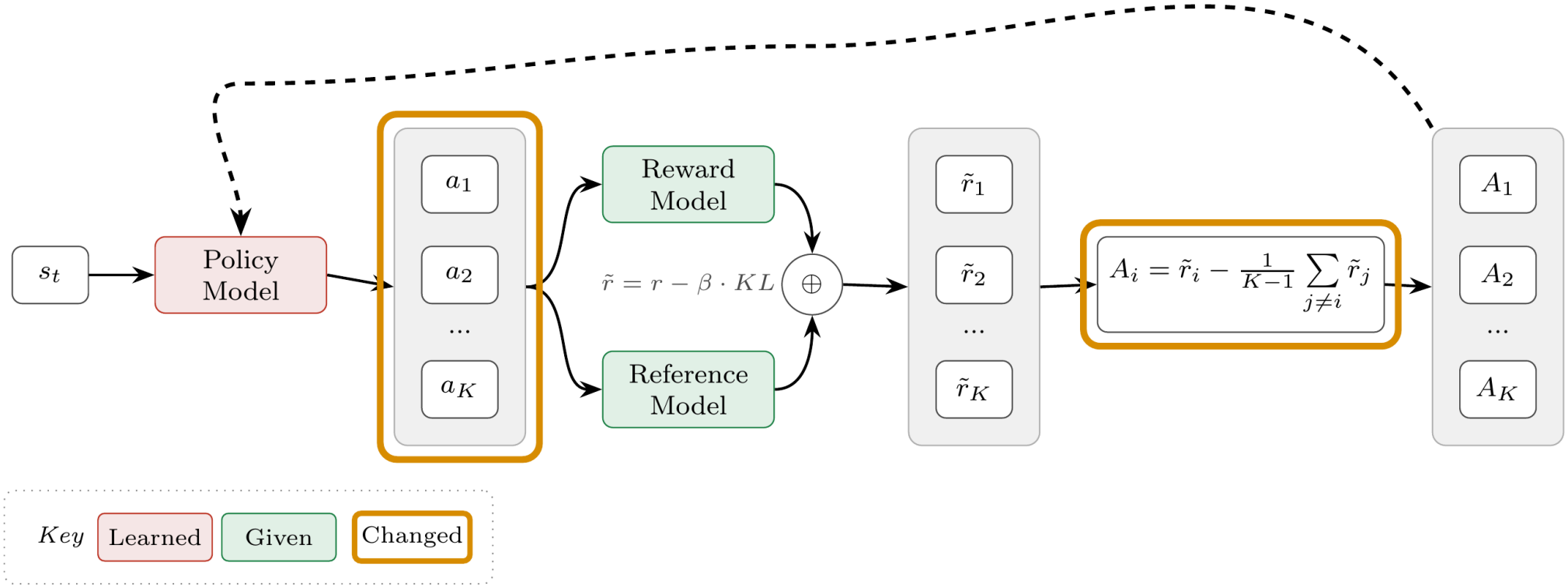
Detail: Like REINFORCE without a critic, this same sequence-level advantage is broadcast to every token in the completion.

In practice, REINFORCE and RLOO are **strong, simple baselines** for RLHF (Ahmadian et al., 2024) (Wang et al., 2024).

Ahmadian et al., 2024

RLOO architecture

OPTIONAL



Changed (orange) vs. the common pipeline of slide 52, i.e. REINFORCE: K responses per prompt; box 3 uses the mean of the *other* responses as baseline.

RLOO worked example

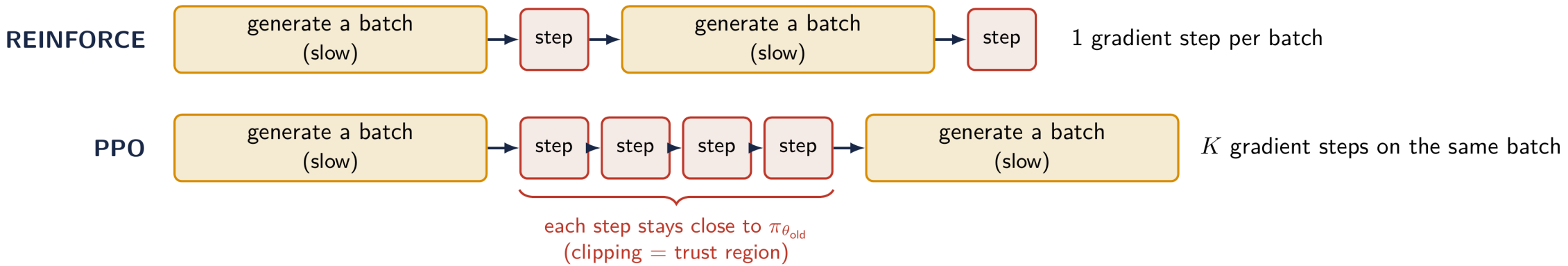
$K = 4$ completions for one prompt, with rewards $[0.8, 0.3, 0.6, 0.5]$:

Completion	Reward	Baseline (avg of others)	Advantage
1	0.8	$(0.3 + 0.6 + 0.5)/3 = 0.467$	$+0.333$
2	0.3	$(0.8 + 0.6 + 0.5)/3 = 0.633$	-0.333
3	0.6	$(0.8 + 0.3 + 0.5)/3 = 0.533$	$+0.067$
4	0.5	$(0.8 + 0.3 + 0.6)/3 = 0.567$	-0.067

Completion 1 (best) gets reinforced. Completion 2 (worst) gets suppressed. Completions 3 and 4 get small updates.

Proximal policy optimization (PPO)

If REINFORCE is so simple, why PPO?

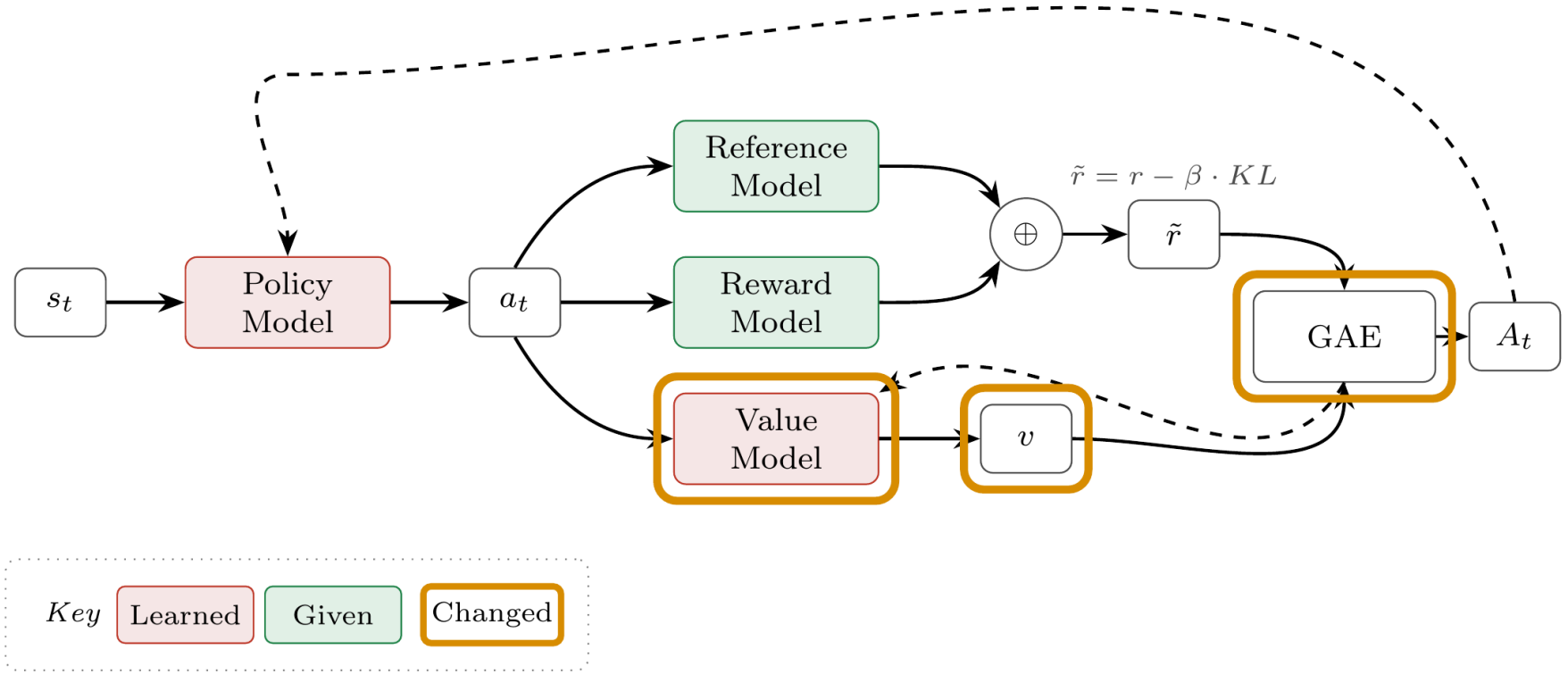


Why? Generating rollouts with an LLM is the slow part. REINFORCE throws each batch away after **one** gradient step. Can we reuse it?

- **Reuse the batch** for K steps (typically 2–4). But after the first step, the policy is no longer the one that generated the data, $\pi_{\theta_{\text{old}}}$.
- **Risk:** a large step can wreck the policy. Keep every step inside a **trust region**, close to $\pi_{\theta_{\text{old}}}$.
- **TRPO** (Schulman et al., 2015) did this with an exact constraint, but **needs expensive second-order optimization**. **PPO** (Schulman et al., 2017) gets a **similar, approximate** effect with a simple **clip**, using two tools: *importance sampling* (to use old data) and *clipping* (to stay close). Both come next.

All methods today are **on-policy**: fresh rollouts from the current policy each round (unlike off-policy methods such as DQN, which replay old experience).

PPO architecture



Changed (orange) vs. the common pipeline of slide 52, i.e. REINFORCE: a second **learned** model, the value model V_ψ; box 3 becomes **GAE** (*generalized advantage estimation*: the value model's predictions + the rewards give one advantage A_t **per token**; details on slide 65); the update is clipped.

PPO tool 1: Importance sampling

Why? PPO reuses each batch for several steps, but after the first step the data no longer comes from the current policy. Importance sampling **corrects for that**, by reweighting each token with a ratio ρ_t.

$\pi_{\theta_{\text{old}}}$ = the policy that **generated this batch**; it changes every batch. Not to be confused with π_{ref}, the **frozen** SFT model used for the KL penalty.

Define the **importance sampling ratio**:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$\rho_t = 1$: both policies agree on this token; $\rho_t > 1$: the current policy now likes it **more**; $\rho_t < 1$: **less**.

Why it works. Fix the state s_t; the action a_t was sampled from $\pi_{\theta_{\text{old}}}$. For any quantity $f(a_t)$ (later: $\hat{A}_t \nabla \log \pi$), weighting by the **same** ratio ρ_t turns an old-policy average into a new-policy one:

$$\mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)}\left[\rho_t\, f(a_t)\right] = \sum_{a_t} \pi_{\theta_{\text{old}}}(a_t \mid s_t)\, \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\, f(a_t) = \mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)}\left[f(a_t)\right]$$

So we can keep using the old batch: just multiply each token's term by its ρ_t.

The surrogate objective

OPTIONAL

Where $\hat{A}_t$ comes from. It is our weight Ψ_t, with the **advantage** chosen (row 5 of the Ψ_t table): "how much better was token a_t than expected?". The hat means *estimated*; PPO estimates it with its value model (GAE, a few slides on).

From the policy gradient to the surrogate, using the old batch:

$$\begin{aligned}
\nabla_\theta J &= \mathbb{E}_{a_t \sim \pi_\theta}\left[\hat{A}_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] && \text{policy gradient, } \Psi_t = \hat{A}_t \\
&= \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}}\left[\rho_t \, \hat{A}_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] && \text{importance sampling (previous slide)} \\
&= \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}}\left[\hat{A}_t \, \nabla_\theta \, \rho_t(\theta)\right] && \rho_t \nabla \log \pi_\theta = \nabla \rho_t \ (\pi_{\theta_{\text{old}}} \text{ is fixed})
\end{aligned}$$

So this is the gradient of the **surrogate objective**, an average over the tokens t of the old batch:

$$J(\theta) = \mathbb{E}_t\left[\rho_t(\theta) \, \hat{A}_t\right]$$

Problem: nothing stops ρ_t from moving far from 1 after a few steps; then the old batch no longer represents the current policy. Next: clip it.

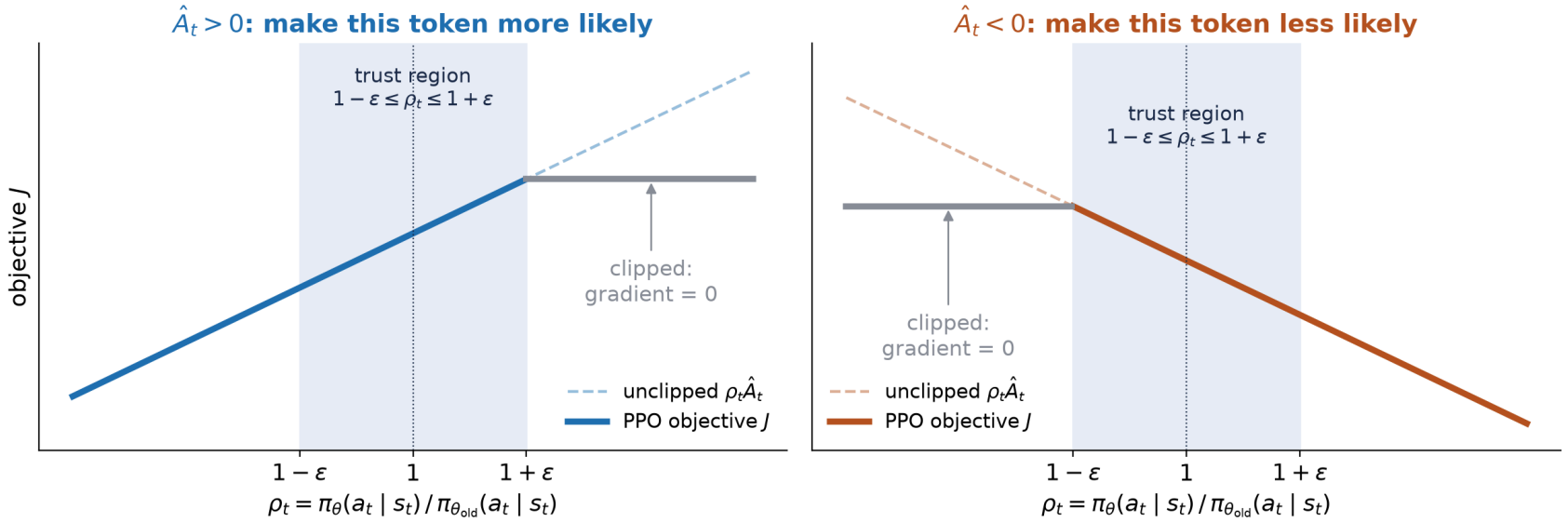
PPO tool 2: The clipped objective

Why? With importance sampling, PPO maximizes $\rho_t \hat{A}_t$ on the old batch. Nothing stops ρ_t from drifting far from 1. Clipping caps how much each token can gain from moving.

$$J(\theta) = \mathbb{E}_t\Big[\min\big(\underbrace{\rho_t(\theta)\,\hat{A}_t}_{\text{unclipped}},\ \underbrace{\text{clip}(\rho_t(\theta),\,1-\varepsilon,\,1+\varepsilon)\,\hat{A}_t}_{\text{ratio kept in } [1-\varepsilon,\,1+\varepsilon]}\big)\Big]$$

- ρ_t: the importance-sampling ratio (slide 60); $\hat{A}_t$: the advantage of token a_t (from the value model)
- ε: typically 0.1–0.2, so the ratio may move by at most 10–20% before clipping kicks in
- $\min$: take the **lower** (more pessimistic) of the two; the next slide shows what that does

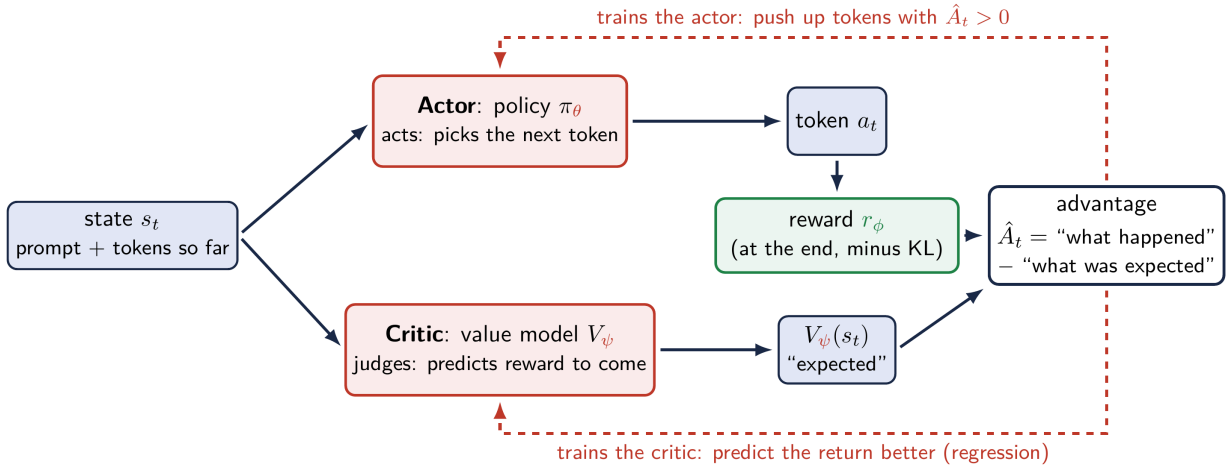
What clipping does



One-sided: clipping only stops the gradient once the token has **already moved enough in the right direction** ($\rho_t > 1+\varepsilon$ for a good token, $\rho_t < 1-\varepsilon$ for a bad one). In the wrong direction it never blocks the correction. If $\hat{A}_t = 0$, no update at all.

Trust region, two ways. TRPO (Schulman et al., 2015): a hard constraint on the **whole distribution**, $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) \le \delta$, solved with second-order optimization. PPO: only clip the ratio of the **sampled tokens**; cheaper and approximate, and it works well in practice.

The critic: a value model



Why? One reward at the end of a response doesn't say **which tokens** helped. A second model can estimate, at every token, how much reward is still to come; then each token gets its own credit.

PPO is an **actor–critic** method: the **actor** π_θ acts (picks tokens), the **critic** V_ψ judges (predicts the reward still to come). Their difference, the advantage, trains the actor.

$$V_\psi(s_t) = \mathbb{E}\left[\sum_{k=0}^{T-t} \gamma^k \, \tilde{r}_{t+k} \,\middle|\, s_t\right] \qquad \text{trained by regression: } \mathcal{L}_V = \tfrac{1}{2}\big(V_\psi(s_t) - G_t\big)^2$$

$\tilde{r}$: shaped reward (reward model at the end, minus the per-token KL); $\gamma = 1$ for language models. V_ψ is usually a copy of the reward model or SFT model with a scalar head.

GAE in one slide

OPTIONAL

Why? PPO needs one advantage **per token**. GAE builds it from the rewards and the value model's predictions.

Step 1, the TD residual: at token t, how much better did things go than the value model predicted?

$$\delta_t = r_t + \gamma\, V_\psi(s_{t+1}) - V_\psi(s_t)$$

Step 2, GAE: add up the residuals of the next tokens, with weights that shrink by $\gamma\lambda$ at each step:

$$\hat{A}_t^{\mathrm{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}$$

λ	what it is	variance	bias
0	one-step TD: $\hat{A}_t = \delta_t$	lowest	highest (trusts V_ψ fully)
0.95	usual default for LLMs	balanced	balanced
1	Monte Carlo: $\hat{A}_t = G_t - V_\psi(s_t)$	highest	none

For language models $\gamma = 1$. If the value model is poor, every $\lambda < 1$ inherits its errors.

Schulman et al., 2015

PPO in practice: one round, four models

One round of PPO:

1. **Generate** completions for a batch of prompts with the policy π_θ
2. **Score** them: reward model r_ϕ, minus the KL penalty to π_{ref}
3. **Advantages** per token: GAE with the value model V_ψ
4. **Update** K times on the same batch (typically 2–4): clipped policy loss for π_θ, regression loss for V_ψ
5. **Repeat** with a new batch; $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$

Two separate regularizers: clipping keeps each step close to $\pi_{\theta_{\text{old}}}$ (this batch); the KL keeps the whole run close to π_{ref}.

Four models in memory

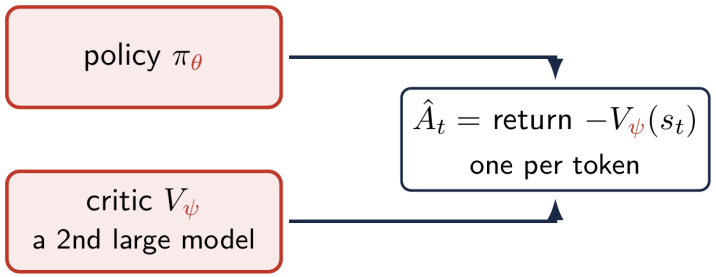
model	learned?
policy π_θ	yes
value model V_ψ	yes
reference π_{ref}	frozen
reward model r_ϕ	frozen

Two of them are full-size LLMs being trained: **memory-hungry**. GRPO (next section) drops the value model.

GRPO & modern variants

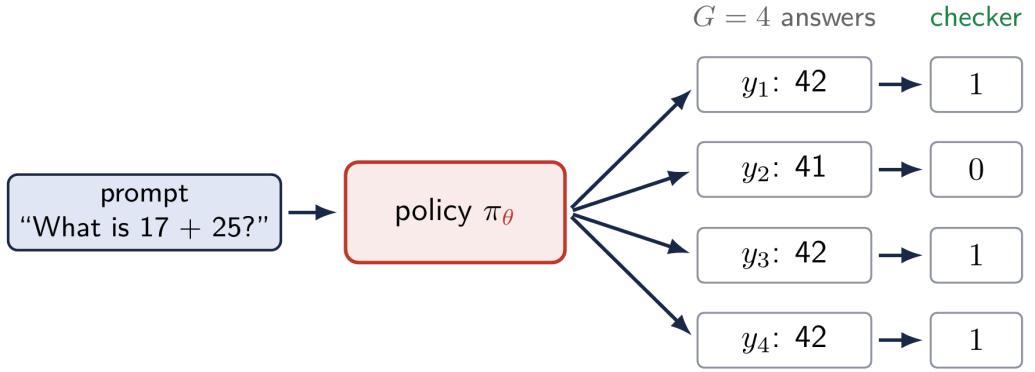
GRPO: PPO for the reasoning era

PPO: the baseline is a learned critic



2 LLMs to train

GRPO: the baseline is the group



mean $\mu = 0.75$
std $\sigma = 0.43$

$\hat{A}_i = \frac{r_i - \mu}{\sigma}$:
$+0.58$ for "42"
-1.73 for "41"

no critic: 1 LLM to train

Why? PPO's critic is a **second large model** to train and keep in memory. GRPO replaces it with the statistics of a **group of answers** to the same prompt, like RLOO.

Group Relative Policy Optimization (DeepSeekMath (Shao et al., 2024)) keeps PPO's **clipped objective** and drops the critic.

- **Simpler and lighter:** one model to train instead of two
- **A natural fit for checkable rewards** (0 or 1): the group tells which answers beat the others
- **Now the default** for RL on language models, popularized by DeepSeek R1

Shao et al., 2024

GRPO: advantage and objective

Advantage. For G answers to the same prompt, with rewards $r_1, \ldots, r_G$ (example on the previous slide):

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

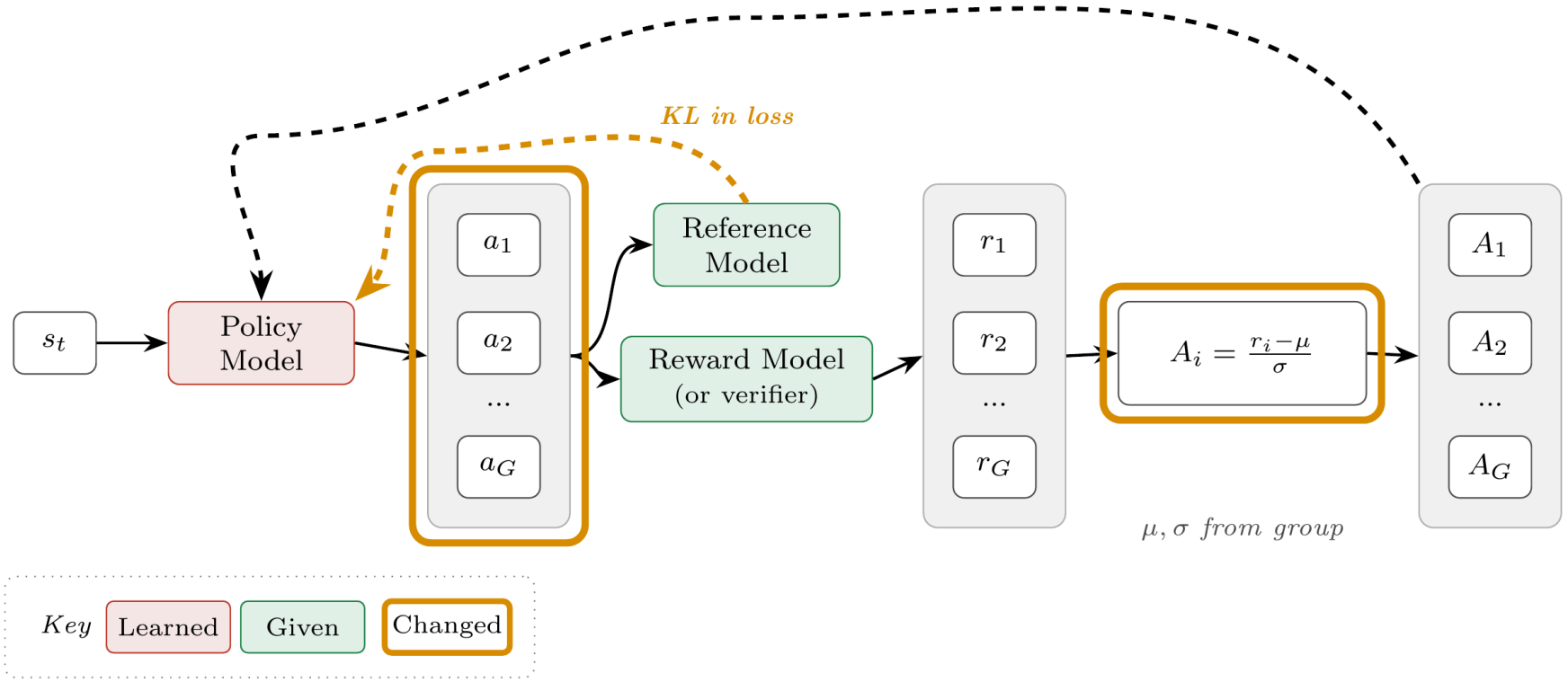
Above-average answers get $\hat{A}_i > 0$, below-average ones $\hat{A}_i < 0$. **Every token** of answer i gets the **same** $\hat{A}_i$.

Objective. PPO's clipped objective, averaged over the tokens of each answer and over the group, with the KL **in the loss**:

$$J(\theta) = \frac{1}{G} \sum_{i=1}^{G} \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left(\min \left(\rho_{i,t}\, \hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\, \hat{A}_i \right) - \beta\, D_{\text{KL},i,t} \right)$$

$\rho_{i,t} = \pi_\theta(y_{i,t} \mid x, y_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$: the ratio is **per token**, the advantage is **per answer**.

GRPO architecture



Changed (orange) vs. the common pipeline of slide 52, i.e. REINFORCE: G responses per prompt; box 3 normalizes within the group, $(r_i - \mu)/\sigma$; the KL moves from the reward **into the loss**; no value model.

PPO, RLOO and GRPO side by side OPTIONAL

	PPO	RLOO	GRPO
answers per prompt	1 or more	K	G
baseline	learned critic V_ψ	mean of the **other** $K-1$ answers	mean of the group, divided by its std
advantage	**per token** (GAE)	per answer: $r_k - \frac{1}{K-1}\sum_{j\neq k} r_j$	per answer: $\frac{r_i - \text{mean}}{\text{std}}$
update	clipped ratio	plain REINFORCE (no clipping)	clipped ratio
KL penalty	in the reward	in the reward (optional)	in the loss (optional)
models in memory	4: policy, critic, ref, RM	3: policy, ref, RM	3, or 2 without KL
typical use	general RLHF	RLHF, simple baseline	reasoning / RLVR (DeepSeek R1)

Same idea, different mechanics: RLOO and GRPO both compare an answer to its **peers**; PPO compares it to a **learned prediction**. Without the division by the std, GRPO's advantage is RLOO's up to a constant factor $\frac{G-1}{G}$.

GSPO: one ratio per answer

OPTIONAL

Why? GRPO's ratio is **per token**, but its advantage is **per answer**. On long answers, one token with a huge ratio can dominate the update.

GSPO uses one ratio per answer: the **geometric mean** of the token ratios,

$$\rho_i(\theta) = \exp\left(\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\log\frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}\right)$$

and clips **that**, like GRPO but at the answer level:

$$J_{\text{GSPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\min\left(\rho_i(\theta)\,\hat{A}_i,\ \text{clip}(\rho_i(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_i\right)$$

Because it is an **average** over tokens, ρ_i stays close to 1 for short and long answers alike, so the same ε works for every length.

Zheng et al., 2025

CISPO: clip the weight, not the gradient

OPTIONAL

Why? PPO's clipping sets a token's gradient to **zero** once its ratio leaves the band; these are often rare tokens that matter. CISPO bounds the weight instead, so every token still learns.

$$J_{\text{CISPO}}(\theta) = \sum_{i,t} \text{sg}\big(\hat{\rho}_{i,t}\big)\, \hat{A}_i\, \log \pi_\theta(y_{i,t} \mid x, y_{i,<t})$$

$$\hat{\rho}_{i,t} = \text{clip}\Big(\frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})},\ 1-\varepsilon^-,\ 1+\varepsilon^+\Big)$$

- $\text{sg}(\cdot)$ = **stop-gradient**: the clipped ratio is used as a fixed number, not differentiated
- So the update is REINFORCE-like ($\hat{A}_i \nabla \log \pi$), with each token's weight **bounded**, never zeroed
- Asymmetric bounds ($\varepsilon^+ > \varepsilon^-$) allow larger reward-increasing updates, which helps exploration

MiniMax Team, 2025

Details that change the objective OPTIONAL

- **Dividing by the standard deviation** gives more weight to prompts where the group's rewards barely differ, which can bias training (Liu et al., 2025)
- **How the loss is averaged matters too:** averaging per response gives each response equal weight, so tokens in long responses count less. Averaging per token gives each token equal weight.

These look like implementation details, but they **change the objective**. Several recent methods change only these normalizations.

Where the KL goes also matters: in the **reward** (PPO, RLOO) it enters the advantage; in the **loss** (GRPO) it stays a separate term, so it doesn't change which answers count as good.

Source: Lambert, Chapter 6.

Key points

- **The RLHF objective** is reward minus β times the KL from a reference model: improve, but don't drift
- **Rejection sampling and best-of-N** use a reward model without RL, and are strong baselines
- **Every policy gradient method** is $\mathbb{E}\left[\Psi_t \nabla_\theta \log \pi_\theta\right]$ with a different weight Ψ_t; all run the **same pipeline** (sample, score, weight, update)
- **Baselines reduce variance without adding bias:** a batch mean (REINFORCE), the other answers (RLOO, GRPO), or a learned critic (PPO)
- **PPO** reuses each batch with importance ratios and **clipping**, and gets per-token credit from a critic: four models in memory
- **GRPO** drops the critic; **GSPO** and **CISPO** change how the ratios are controlled
- **Normalization and where the KL goes** look like details, but they change the objective

The big picture: in practice, the quality of the **data and the reward** matters more than the choice of algorithm; the algorithm mostly decides stability, speed and memory.

All core RL losses on one page: Lambert's RL cheatsheet.

Lab: learning from a checker

Reasoning models

Why? The lab trains with a **checker** instead of a reward model. This is how reasoning models (o1, DeepSeek R1) are trained.

- **RLVR** (slide 18): the reward is 1 if the extracted final answer is correct, 0 otherwise
- Trained this way, models learn to write long **reasoning traces** before answering, and accuracy keeps improving with more RL and more thinking (slide 17)

Measuring success: pass@k = the chance that **at least one** of k tries is correct. For one question:

- n = number of responses sampled · c = how many of them are correct · k = tries allowed ($k \leq n$)
- Example: $n = 8, c = 3$. pass@1 $= 3/8$; pass@8 $= 1$ (one of the 8 is right).

The unbiased estimate from n samples with c correct, choosing k of them without replacement:

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \qquad \text{(the fraction of } k\text{-subsets with no correct answer, subtracted from 1)}$$

Source: Lambert, Chapter 7.

Guo et al., 2025

Common practices in reasoning training OPTIONAL

- **Difficulty filtering:** a prompt solved always, or never, gives **no gradient**, because every response in its group gets the same reward. Keep prompts the model sometimes solves. (On slide 68, if all 4 answers were “42”, every advantage would be 0.)
- **Remove zero-advantage groups** during training, and refill the batch
- **Drop the KL penalty:** a checker is harder to exploit than a reward model
- **Asymmetric clipping** ($\varepsilon^+ > \varepsilon^-$, slide 73), to keep exploring
- **Format and language rewards:** reward a valid reasoning block and a consistent language
- **Length control,** against overthinking
- **Normalize across the batch,** not per group (slide 74)

An open question: does RL **discover** new abilities, or **elicit** what pretraining already learned?

Source: Lambert, Chapter 7.

Lab: what you'll do

The task: "how many times does the letter *n* appear in this text?" No good answers to imitate, no comparisons: **only a checker.** Qwen3-0.6B gets it right about a third of the time.

1. **The task and its checker** (Part 1)
2. **The model can already do it, sometimes** (Part 2, Exercise A): pass@1 and pass@8, with intervals across prompts
3. **From rewards to a gradient** (Part 3, Exercises B and C): group advantages, which groups give no gradient, and the policy-gradient loss
4. **Train with GRPO** (Part 4): 20 minutes on a free T4; watch reward, contrast, length and KL
5. **What changed** (Part 5): does RL teach a new skill, or turn "sometimes" into "usually"?

Open the lab: lab-03-grpo.ipynb in Colab. Work in pairs.

Source: Lambert, Ch. 6, Suggested experiments.

Before next session

Reading, to review today's session

- **Lambert, *RLHF*:** Chapter 6 (policy gradients), and the one-page RL cheatsheet
- **Lambert's course:** Lecture 3 (the math) and Lecture 4 (the implementation)

References (1/2)

Ahmadian, A., Cremer, C., Gall\''e, M., Fadaee, M., Kreutzer, J., et al. "*Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs*." *arXiv preprint arXiv:2402.14740*, 2024.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., et al. "*Training a helpful and harmless assistant with reinforcement learning from human feedback*." *arXiv preprint arXiv:2204.05862*, 2022.

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., et al. "*Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning*." *arXiv preprint arXiv:2501.12948*, 2025. [link]

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training*." *arXiv preprint arXiv:2411.15124*, 2024. [link]

Lattimore, T., and Szepesvári, C. "*Bandit Algorithms*." *Cambridge University Press*, 2020. [link]

Liu, Z., Liu, C., Gao, W., Liang, Z., Liu, T., et al. "*Understanding R1-Zero-Like Training: A Critical Perspective*." *arXiv preprint arXiv:2503.20783*, 2025.

MiniMax Team. "*MiniMax-01: Scaling Foundation Models with Lightning Attention*." *arXiv preprint arXiv:2501.08313*, 2025.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., et al. "*WebGPT: Browser-assisted question-answering with human feedback*." *arXiv preprint arXiv:2112.09332*, 2021.

OpenAI. "*Introducing OpenAI o1-preview*." *OpenAI Blog*, 2024. [link]

References (2/2)

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. "*High-Dimensional Continuous Control Using Generalized Advantage Estimation*." *arXiv preprint arXiv:1506.02438*, 2015.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. "*Trust Region Policy Optimization*." *International Conference on Machine Learning*, 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. "*Proximal Policy Optimization Algorithms*." *arXiv preprint arXiv:1707.06347*, 2017.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., et al. "*DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*." *arXiv preprint arXiv:2402.03300*, 2024.

Sutton, R., and Barto, A. "*Reinforcement Learning: An Introduction*." *MIT Press*, 2018. [link]

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., et al. "*Llama 2: Open foundation and fine-tuned chat models*." *arXiv preprint arXiv:2307.09288*, 2023.

Wang, Z., Dong, Y., Delalleau, O., Zeng, J., Shen, G., et al. "*HelpSteer2-Preference: Complementing Ratings with Preferences*." *arXiv preprint arXiv:2410.01257*, 2024.

Zheng, Z., Guo, D., and others. "*Group Sequence Policy Optimization*." *arXiv preprint*, 2025.